
[PRD] [TF-273]: Policy workflows on Terraform Stacks (Phase 1)

Summary: This document captures the problem statement and requirements for the first phase of policy capabilities on stacks using tfpolicy.

Created: Jan 8, 2026

Product: Terraform

Contributors: Brian.Earwood@ibm.com, christina.chung@ibm.com, Vanshika.Munjal@ibm.com

Projects: [Terraform Policy](#)

Stakeholders: N/A

Status: WIP | **In-Review** | Approved | Obsolete

Owners: v.paul@ibm.com

Approvers: Nicoleta.Popoviciu@ibm.com, puttaraju.ks@ibm.com, Jon.Johnson@ibm.com, Liam.Cervante@ibm.com, katy1@ibm.com,  omar.i@ibm.com

RFC: N/A

NOTE: This [document](#) is managed by, [Hermes](#) and this header will be overwritten using document metadata.

JIRA: <https://hashicorp.atlassian.net/browse/RDPR-179>

This document covers the first phase of policy authorship, testing, administration, and evaluation workflows as they relate to Terraform Stacks.

The discovery for this initiative flows from the interviews conducted with customers who use policy as it exists on our platform today, and from research on how customers intend to use Stacks.

Background (can be skipped)

Terraform Stacks became generally available at HashiConf 2025, and was in public beta for about a year prior to that. During this period, Stacks has steadily moved towards becoming enterprise-ready, with its focus on scalable APIs and sophisticated UX.

However, it still does not have certain capabilities available on workspaces today that block its adoption by some of our largest customers. Policy is one of these capabilities.

Read more about the Stacks GA payload here: [\[TF-229\] Stacks General Availability \(Cloud\) - PRD](#)

Customers who are interested in Stacks but blocked by lack of policy support:

JPMC, BoFA, HCA, Blackstone, Banco Bradesco, Elevance Health

Existing workflow and the planfile dependency

There have been explorations of Sentinel's integration with Stacks, typically aimed towards getting around the limitation of Sentinel's dependency of the workspace plan file, [which cannot be mapped 1:1 with a Stack plan file](#).

Policies in Sentinel and OPA read values from the plan file (and potentially the state and config files) and make assertions based on information there. [Early \(very high-level\) technical investigations](#) suggested that even with modifications to Sentinel, customers would need to rewrite their policies to see them applied to these new workloads/workflows.

Deferred changes

Terraform Stacks also introduce the concept of 'Deferred Changes', which handle scenarios where certain Components can only be planned after retrieving values resolved later in the Run. Since this generates what we refer to as a 'partial plan', it further complicates any hypothetical static planfile evaluation conducted by Sentinel (or OPA).

These requirements are some of many that culminated in the development of a new PaC framework: **Terraform policy** (also referred to as **tfpolicy**), a tool that is directly plugged into Terraform, and leverages the power of the Terraform provider ecosystem to obtain information available to Terraform at any point in time.

The direct targeting of resources, modules, and providers agnostic of the underlying workload (workspace, stack), along with the ability to 'speak live' to Terraform will allow customers to write a policy once, and have it applied at any stage of the infrastructure lifecycle.

Problem

The broader problem of evaluating infrastructure against security rules is not new, and will not be covered in this document. The problems highlighted below are specifically those that customers will encounter when applying policies to Stacks.

P1: Mapping of policy evaluations to appropriate Stack constructs

Customers are able to evaluate policies today by selecting workspaces, either directly or using grouping constructs like Projects and Organizations. Similarly, our policy administration workflow must allow customers to intuitively map policy sets to the right constructs.

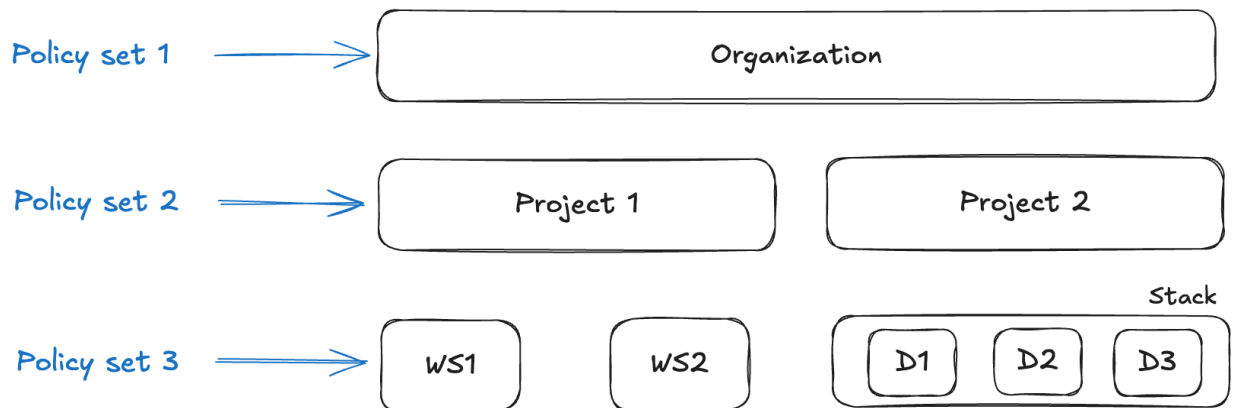


Figure 1: Policy sets applied to various levels in our HCP TF hierarchy

Customers use a Stack to repeat infrastructure across multiple environments, accounts, etc. This is done by using multiple ‘Deployments’ that carry the same set of ‘Components’. Read more about Stacks constructs [here](#).

A single workspace maps most closely to a ‘Deployment’, as a Deployment is the unit that is planned and applied in its entirety, whenever there are any changes to the Stack.

P2: Ensuring reusability of policies across workspaces and Stacks

Platform Engineers typically write policies to meet a set of controls provided by their security counterparts. The resources, modules and providers that need to meet these policy requirements are the same irrespective of whether they are created within the context of a workspace or a Stack.

Therefore, a customer will expect to reuse the same policies across workspaces and Stacks.

P3: Preserving common policy enforcement patterns in context of a Stack

Often, customers introduce a new policy in ‘advisory’, assessing its impact across a certain set of workspaces. This also allows developers to proceed with work while being made aware of a policy they must eventually comply with.

Once a certain amount of time has passed or a pre-determined date to update the policy has arrived, the PE updates the policy in the policy set to be ‘mandatory’. **Policy sets therefore contain a mix of enforcement levels and target a group of workspaces/projects.**

Customers may also want to be able to override policy failures that occur on this policy set.

Customers: Duke Energy, SPH Media, Humana

Context: Enforcement levels

Advisory: Updates to infrastructure that violate a policy result in warnings after a Plan, and do not block an Apply

Mandatory (overrideable) or soft-mandatory: Updates to infrastructure that violate a policy result in failing the Run, but a user with appropriate permissions can override the failure and allow an Apply.

Mandatory: Updates to infrastructure that violate a policy result in failing the Run

Changes to the enforcement behavior in tfpolicy are documented here: [\[TF-274\]](#) [\[tfpolicy\]](#)
[Enforcement levels and overrides](#)

P4: Ensure ‘Deferred changes/Partial plan’ evaluation is consistent and intuitive

Deferred changes involve multiple Plans and Applies, and customers expect that a Deployment with deferred changes will still obey security rules of the organization. Therefore, the policy workflow must account for the new resource attributes generated during this process and evaluate those just like they would for a newly triggered plan.

Personas

Platform Engineers (PEs) are tasked with codifying compliance controls that are to be applied throughout the infrastructure and application estate they manage.

In this document, we may reference **Policy Authors** and **Policy Administrators**, who are responsible for writing and testing policies, and managing policies respectively. It is usually the PE who dons these hats.

Application Developers specify the modules and providers used to provision resources within their Terraform config, and expect that organizational security rules will automatically be applied wherever necessary.

Read more about personas and the JTBD in the context of Terraform Stacks here: [\[Meta-PRD\]](#)
[Terraform Stacks](#)

High-level JTBD

When **managing infrastructure at scale** across multiple regions, environments, and accounts I want to **ensure that any changes** across these repeatable units **happen in accordance with security rules** declared by our organization

So that my organization **remains secure** when introducing widespread changes.

Critical User Journeys

- As a **Policy Author**, I want a **mechanism to write and test policies easily** so that I can apply those policies to infrastructure and **maintain the security posture of my organization**.
- As a **Policy Administrator**, I want to **evaluate the infrastructure in my Stack with the right security rules** so that that the **infrastructure is in line with the security posture of my organization**.
- As an **Application Developer/Platform Engineer**, I want to **observe a rich set of diagnostics** during policy evaluations so that I can **make appropriate changes to my code** and make sure it is compliant with the rules set by the Policy Administrators.

- As a **Platform Engineer**, I want the **ability to override policy failures** so that I can **unblock Application Developers and maintain productivity in special cases** where a policy failure is temporarily acceptable.

Requirements

R1. Authorship: Writing and testing policies

R1.1. Basic policy authorship and testing as it relates to a Stack will remain the same.

Policies against resources, modules and providers will be written and tested the same way as they are for workspaces.

Policy authorship: [\[STL-005\] Terraform Policy Language UX](#)

Policy test framework: [\[STL-004\] Terraform Policy Test Framework](#)

For example, a policy against a resource is agnostic to whether it lives in a workspace or Stack:

```

❑resource_policy "aws_ebs_vol" "example" {
    enforce {
        condition = attrs.encrypted == true
    }
}
❑

```

Acceptance criteria

Any changes introduced to the DSL and Test Framework as part of the [tfpolicy private beta](#) will be available to the user even if the eventual target for the policy is a Stack.

Considerations

Modules referenced inside a Component will have different addresses than when they are referenced in the workspaces. In these scenarios, we expect the Policy Author will account for this difference.

R1.2. A policy author will be able to reference Stack and Deployment metadata from the DSL

A policy author may want to check certain Stack metadata such as the name or a tag, and will need a way to reference these using the DSL.

The following example is for illustrative purposes only, the exact syntax may be different.

```

❑resource_policy "aws_ebs_vol" "example" {
    filter = !core::contains(meta.tfe_stack.deployment.tags, "dev")

    enforce {
        condition = attrs.encrypted == true
    }
}

```

```
}  
}
```

□Acceptance criteria

AC1. The policy author must be able to reference Stack metadata such as name (or tags) in future.

AC2. The policy author must be able to reference Deployment metadata such as name (or tags) in future.

R2. Administration: Managing policies on Terraform Cloud/Enterprise

R2.1. Introduce a Stack as a target in the Policy Set CRUD workflow

Today, Policy Set workflow allows a user to target an organization, project(s) or workspaces. A Stack will be introduced as a new target. The policy set will target all the Deployments in the Stack.

Acceptance Criteria

AC1. In the Policy Set CRUD workflow, under 'Scope of Policies', introduce an option of 'Policies enforced on Stacks'. When this option is selected, the user is then presented with a drop-down of all the Stacks available in that organization. They can select one or many from this list.

AC2. When options of 'Global' or 'Select Projects and Workspaces' are selected, users must be informed that these selections do not include any Stacks that may have been created in those projects.

AC3. Evaluation Stages will not be shown as Plan time is the default and only selection.

R2.1 - Stretch: Introduce Deployments in a Stack as targets

Since Stacks are used to manage multiple similar environments using Deployments, one can assume that different Deployments would sometimes need to follow different security rules.

Acceptance Criteria

Once a Stack is selected as a target, the user is presented with the option to select or exclude specific Deployments, similar to the way workspaces can be included or excluded after a Project has been selected.

R2.2. Maintain current override permissions

In general, when a policy evaluation fails, the Run fails, and an Apply is blocked. However, a user with the appropriate permissions can override this failure and allow the Run to proceed.

Acceptance Criteria

A user with the 'Manage override permissions' at the organization or project level has those permissions extended in the context of a Stack.

R3: Evaluation: Applying policies and observing results

We will only support plan-time policies in this phase. Pre-plan and Apply-time policies will be incorporated once those workflows are hardened in the context of workspaces.

R3.1. A policy set will be evaluated against each deployment in a Stack

As of the current behavior, each deployment in a Stack is planned independently of the others. This plan can be triggered through the usual interfaces of UI, CLI, and VCS.

A policy evaluation will be triggered whenever a new plan is triggered on a Deployment.

Acceptance Criteria (general behavior)

AC1. A policy evaluation will be triggered whenever a new plan is triggered on a Deployment.

AC2. A policy failure will result in an Apply being blocked, irrespective of whether `auto-approve` is enabled on the Deployment or not.

AC3. All resources whose attributes are known at plan time will be evaluated. Warnings will be shown for those that are not known.

Acceptance Criteria (UX)

AC1. (UI) A new panel with diagnostics will be presented on the Deployment Run screen and display the results of policy evaluations.

AC2. (UI) This panel will remain consistent with other Run events.

AC3. (CLI-driven) A summary of policy evaluations will be presented with an overall Pass/Fail count for each deployment. The user will then be presented with a link to the UI to view detailed diagnostics there.

Considerations

There will be no policy evaluations that relate to how one stack may be linked to another. We are yet to understand whether or what kinds of policies would be useful here.

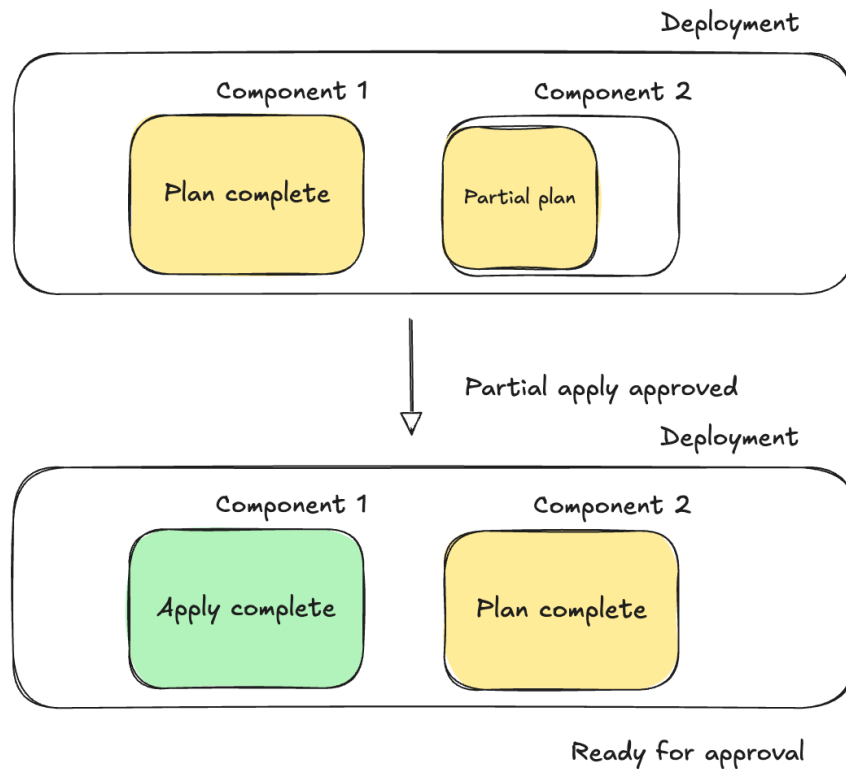
R3.2. Deferred changes will be treated as another plan-time evaluation

Deferred changes result in automatically re-initiating a Plan on a part of a Deployment that was waiting on information that is determined later in the planning process.

Note: Policy pass/fails in the following sections refer to mandatory policies.

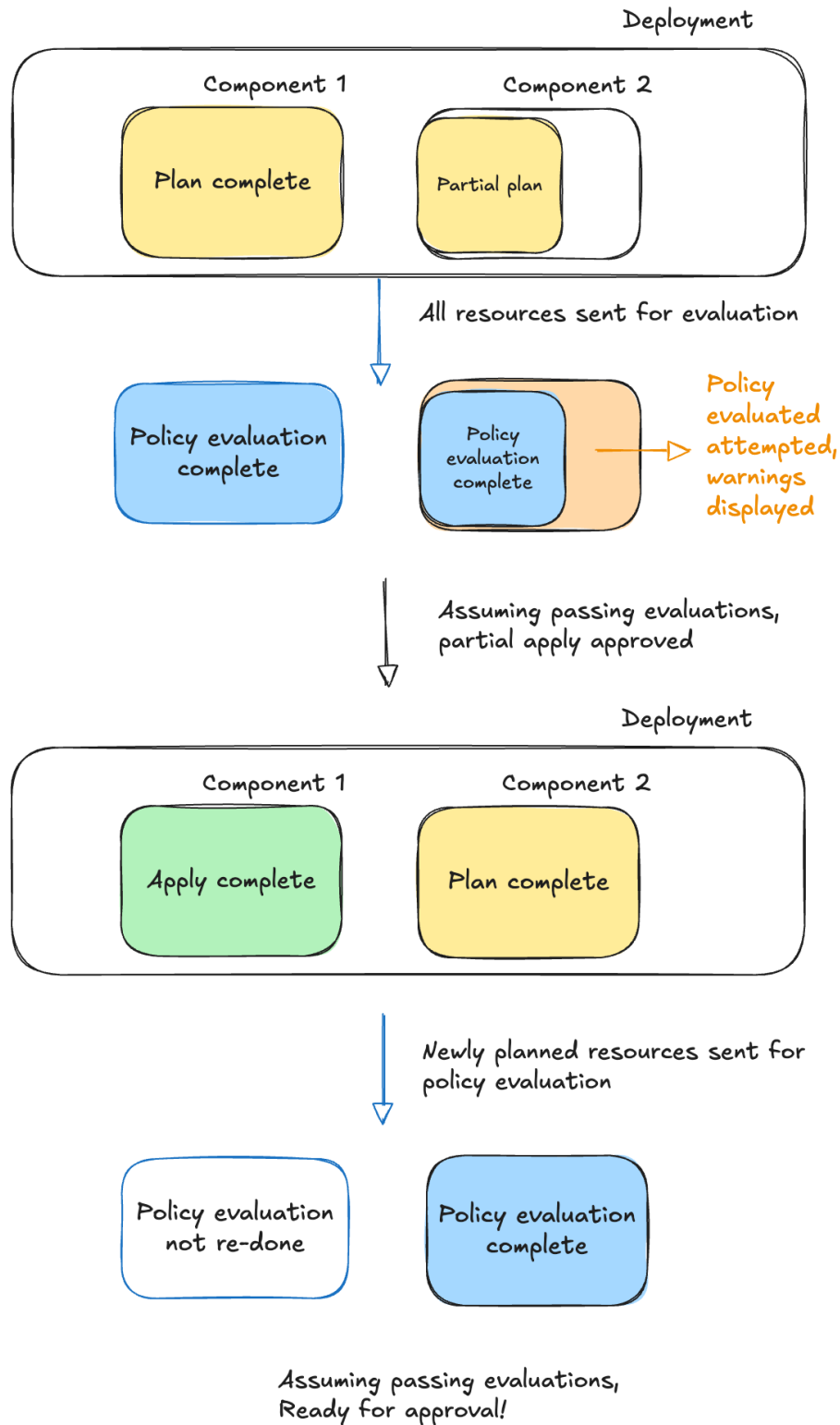
Scenario Analysis (can be skipped)

Consider the following scenario with one deployment, but with two Components, one of which generates a partial plan till the other is applied.



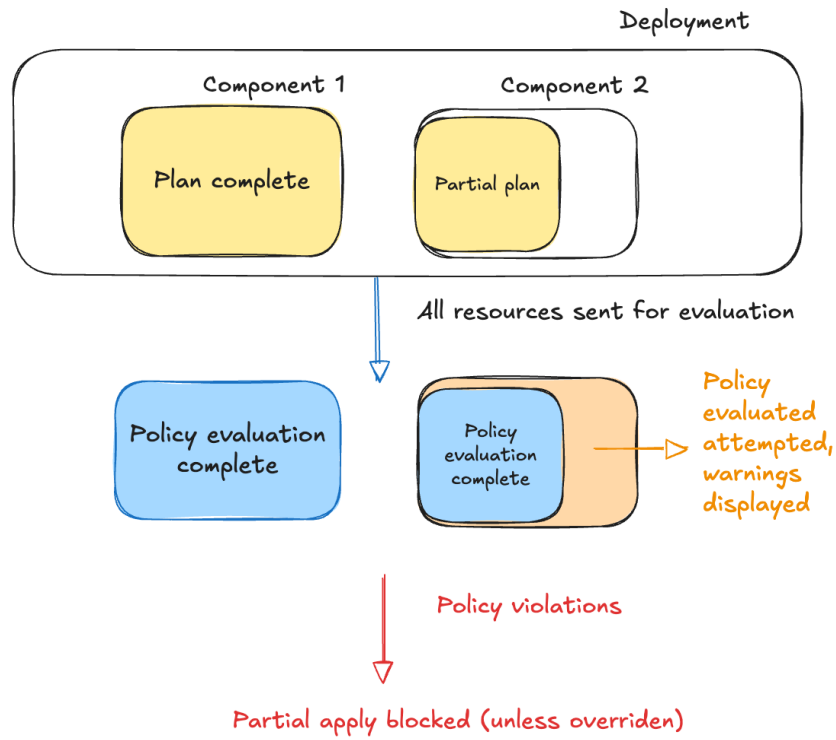
The 'Happy Path'

This is when a partial plan is generated, but all the policy evaluations on known attributes of resources pass. Warnings are generated for unknown values (this is the same as the 'Known After Apply' behavior in Terraform policy today), but do not block an Apply.

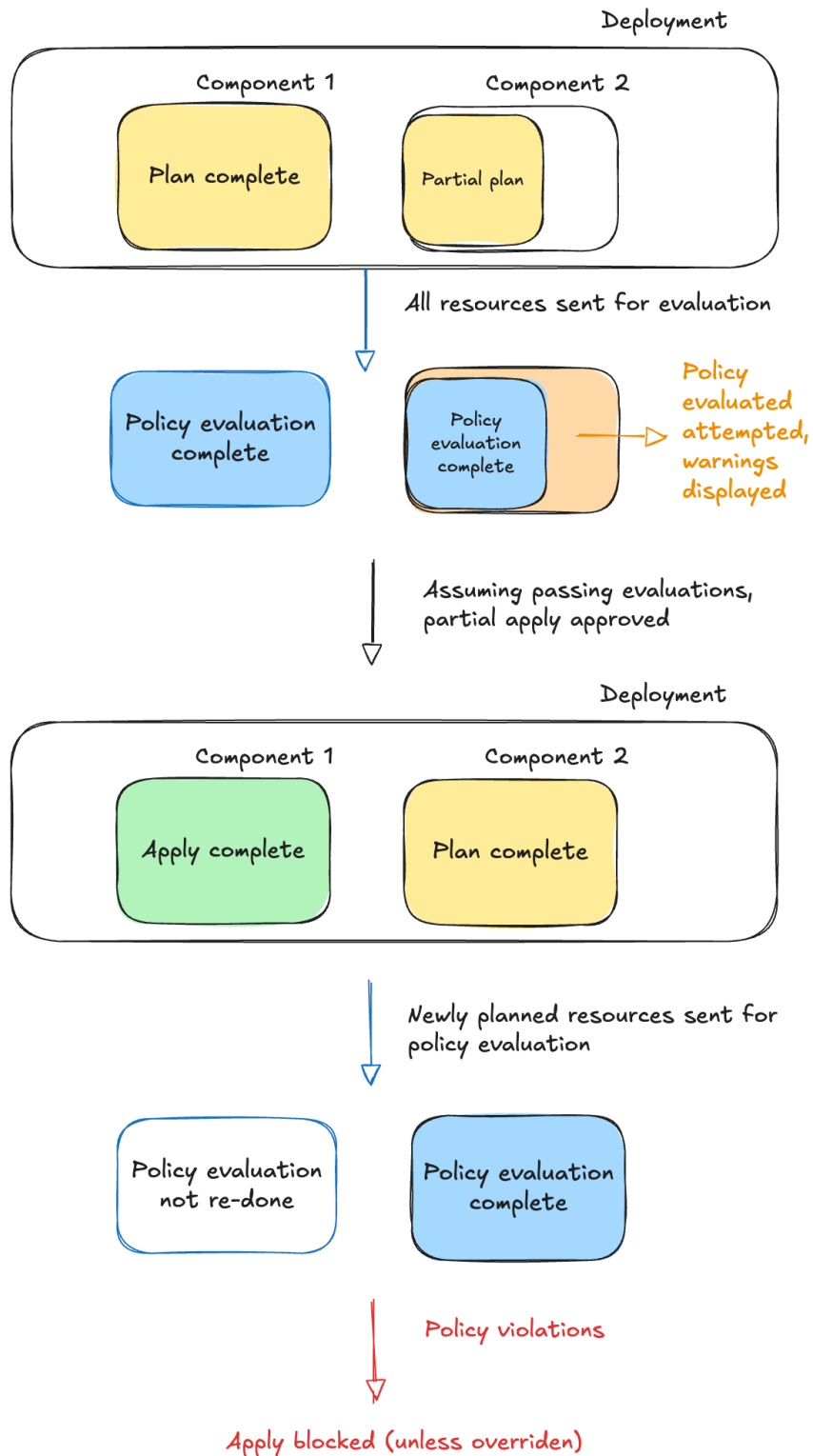


In the event of policy failures (non- 'Happy Path'):

- a. This can happen at an early stage when only partial plans are generated. Even a partial Apply is blocked. The second Component is not planned, as it would be planned with values that are not compliant with the organization's security rules.



- b. If the first set of evaluations pass and the second fails, the next Apply is blocked. This means that infrastructure is partially provisioned until the policy failure is resolved.



Acceptance Criteria (general behavior)

AC1. All attributes known at the time of an initial plan will be subject to an evaluation, and the diagnostics will be presented for the same (if applicable).

AC2. A partial Apply can be approved if the policy evaluations pass. Warnings will be presented to the user for unknown values.

AC3. Once a re-plan is triggered on a Component whose resources are now ready to be planned, the new 'known' resources will now be evaluated against the same policies as before. The subsequent results will be presented in place of the Warnings.

Considerations

In a scenario where the previously partial plan results in policy violations (non-Happy Path, (b)), the entire **Deployment will only be partially applied until the policy violation has been addressed or overridden.**

R3.3. Overrides will be presented to users with permissions

In the scenario where policy failures occur, a user with the appropriate permissions will be able to override the failures and enable an Apply.